On Checking the Conformance between iStar and BPMN Models

Duc-Quyen Nguyen, Minh-Phuc Hoang, Huy-Hung Dao, and Duc-Hanh Dang*
Faculty of Information Technology, VNU University of Engineering and Technology
VNU Town, Hoa Lac, Hanoi, Vietnam
{25025048 | 22028059 | 22028033 | hanhdd}@vnu.edu.vn

***Abstract*—Within model-driven engineering, goal and process models are utilized to capture complementary intentional and operational views of a system. However, checking conformance between these two kinds of models remains challenging due to the underlying semantic gap between them. This paper introduces a method to check the conformance between two models expressed in the standard modeling languages iStar and BPMN, respectively. The method relies on a unified model that integrates iStar and BPMN, providing shared semantics for the two languages. Such an operational semantics is characterized by execution scenarios, generated as sequences of state transitions, where each current state is obtained by composing the organizational state and the domain state. To represent such a current state, we introduce a modeling language named CSL (Composite State Language). By evaluating the possible scenarios generated through model finding on the unified model, we check the conformance between the two models. A supporting tool is implemented as a plugin for USE, an OCL-based modeling tool. Experiments on typical case studies illustrate and evaluate the method, indicating its potential applicability in practice.**

***Index Terms*—GORE; iStar; BPMN; CSL; OCL; Conformance Checking**

I. Introduction

Model-driven engineering (MDE) places models at the center of the software engineering process and treats them as first-class engineering artifacts [1]. Within MDE, a system is often described by multiple complementary models that capture different concerns and levels of abstraction. Goal models represent stakeholders, intentions, and rationale, thus explaining *why* a system is needed, while process models describe activities, responsibilities, and execution order, thus explaining *how* those intentions are operationalized [2]. The intentions expressed in a goal model are expected to be realized by the behavior described in the corresponding process model. Establishing traceability and analyzing consistency between the two models help to relate the goals to their process-level realization and detect divergence as the models evolve [2], [3]. Checking conformance between goal and process models is challenging for several reasons. First, the two models are developed at different levels of abstraction, since one goal may be realized by multiple activities and one activity may contribute to several goals [3], [4]. Second, they are often developed independently, making direct correspondences difficult to maintain as the models evolve [2], [4].

*Corresponding author.

Existing execution-oriented approaches address some of these challenges from different perspectives. Koliadis and colleagues [5], [6] relate process execution to goal satisfaction through activity effects and fulfillment conditions, but their analysis remains largely methodological and relies on manually specified annotations. Our previous work [7] synchronizes execution scenarios through a common UML-based model, but UML does not explicitly capture the organizational structures needed to relate goal and process models. Caballero-Villalobos et al. [3] transform process and goal models into transition systems and evaluate their combined behavior, but their compliance criteria focus on the reachability and stability of quality satisfaction.

To address these limitations, we propose a method for integrating iStar and BPMN models into a unified model, forming shared semantics for the two languages. We define the unified model and a mechanism for representing and generating its execution scenarios as sequences of state transitions. Each state combines the organizational state and the domain state and is represented using the Composite State Language (CSL). Scenarios are generated from the unified model through model finding and evaluated against the intentions expressed in iStar. Based on these scenarios, the conformance is classified as strong conformance, weak conformance, or non-conformance, while scenarios that violate required intentions are identified as risks. The method is supported by a plugin for USE, an OCL-based modeling environment. Experiments are conducted on typical case studies, including *ProposalReview*, to illustrate the method and indicate its potential applicability in practice.

This paper makes three main contributions: (i) a mechanism for defining and representing a unified semantics for goal and process models; (ii) a method for checking conformance between these two types of models through execution scenarios; and (iii) a supporting tool based on USE together with experiments that illustrate and evaluate the proposed method.

The remainder of this paper is organized as follows. Section II presents the motivating example and background. Section III outlines our proposed method. Section IV explains the unified model of iStar and BPMN. Section V describes our method for checking conformance between the two models. Section VI presents the tool support and discussion. Section VII reviews related work. Finally, Section VIII concludes the paper and outlines future work.

II. Motivating Example and Background

We consider *ProposalReview* [8], a simplified proposal-handling case. The *Proposal Manager* prepares, revises, and validates proposals, while the *Customer* reviews the current version and either accepts it or requests further changes before finalization.

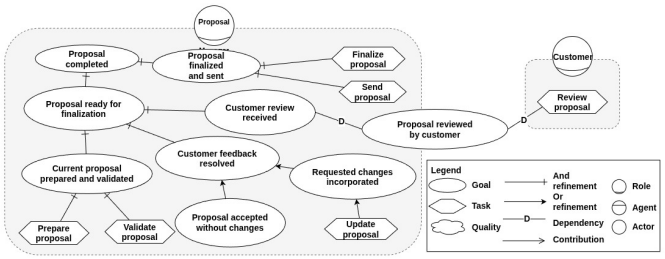


Fig. 1: The iStar model of goals for *ProposalReview*.

Figure 1 presents the intentional view, showing the actors involved, the goals they pursue, and the tasks used to realize those goals [9]. Refinement captures how lower-level intentions contribute to the satisfaction of a higher-level one. In the proposal case, readiness for finalization requires both preparation and validation, illustrating AND-refinement, where all refined intentions must be satisfied. Feedback resolution illustrates OR-refinement, since it can be achieved either by accepting the proposal without changes or by incorporating the requested revisions. Dependencies capture situations in which one actor relies on another to achieve an intention. In this case, the *Proposal Manager* depends on the *Customer*'s review and feedback to resolve the proposal review.

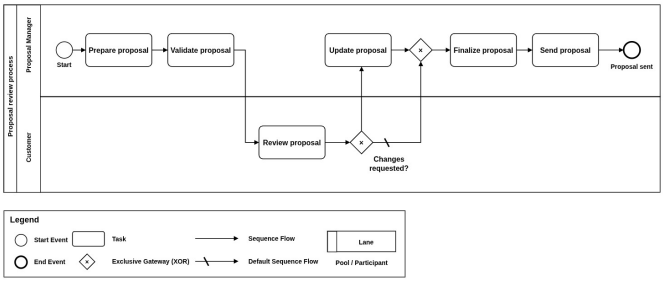


Fig. 2: The BPMN process model for *ProposalReview*.

Figure 2 presents the corresponding process view, in which execution is represented through activities, events, gateways, and sequence flows [10]. In *ProposalReview*, activities capture the preparation, validation, review, revision, finalization, and delivery of the proposal. Events mark the process boundaries, while the gateway directs execution according to the Customer's review. A request for changes therefore leads to a revised proposal, whereas acceptance allows the process to continue toward finalization.

The two models are considered consistent when the process structure provides realizations for the intentions represented in the goal model. For instance, BPMN activities such as *Prepare proposal* and *Validate proposal* contribute to the corresponding goals in iStar, while an alternative gateway can reflect an OR-refinement between alternative ways of satisfying a goal [4], [6]. In this case study, the iStar and BPMN models can therefore be regarded as consistent. However, modifying a previously validated proposal changes the content on which the earlier validation was performed. In practice, the revised proposal must therefore be validated again before it can be considered valid. Consequently, updating the proposal makes the validation goal no longer satisfied until a new validation is performed. The BPMN model nevertheless allows execution to proceed directly from the update to finalization and sending. The process can thus reach completion even though a goal required by the iStar model is no longer satisfied. This exposes a conformance problem that cannot be identified from structural consistency alone.

III. Overview of the Proposed Method

The proposed method checks conformance between an iStar goal model and a BPMN process model through the three steps illustrated in Fig. 3.

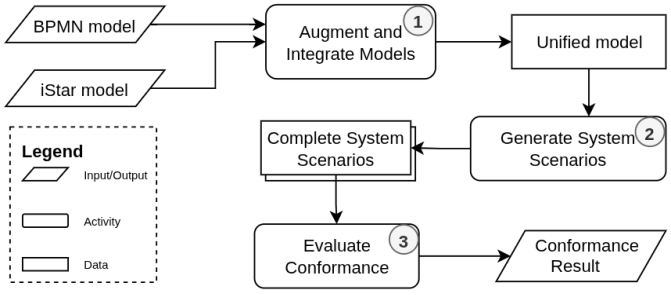


Fig. 3: Overview of the proposed method.

The first step identifies the shared organizational and domain state and represents it using a CSL model. The organizational concepts of iStar and BPMN are then aligned with their corresponding CSL elements. To make their semantics state-dependent, conditions are added to the leaf elements of the iStar model, while BPMN flow elements are augmented with pre- and postconditions and sequence flows with guards. The result is a unified model that combines the intentional semantics of iStar, the execution semantics of BPMN, and the shared state represented by CSL. Next, the unified model is executed to generate system scenarios. Starting from admissible initial states, the method explores the possible executions of the BPMN model while updating the shared CSL state and the corresponding iStar evaluations. This produces the set of complete system scenarios considered by the analysis. Finally, these scenarios are evaluated with respect to the root goals of the iStar model. Their outcomes are aggregated to classify the conformance of the integrated models as strong conformance, weak conformance, or non-conformance. The analysis also determines whether the models are risky by checking whether any complete scenario ends with a violated root goal.

IV. Unified Model of iStar and BPMN

This section explains how a unified model integrating iStar and BPMN is defined. We first extend iStar and BPMN with state-dependent specifications expressed in OCL [11]. We then define CSL as a language for representing the current state of an execution of the unified model.

A. Extension of the iStar Model

To represent the state of a goal model, we extend the iStar metamodel [9] with state-dependent constraints expressed through the `Condition` concept, shown in Fig. 4. The extension also builds on the temporal interpretation introduced in [12] and is adapted here for conformance checking.

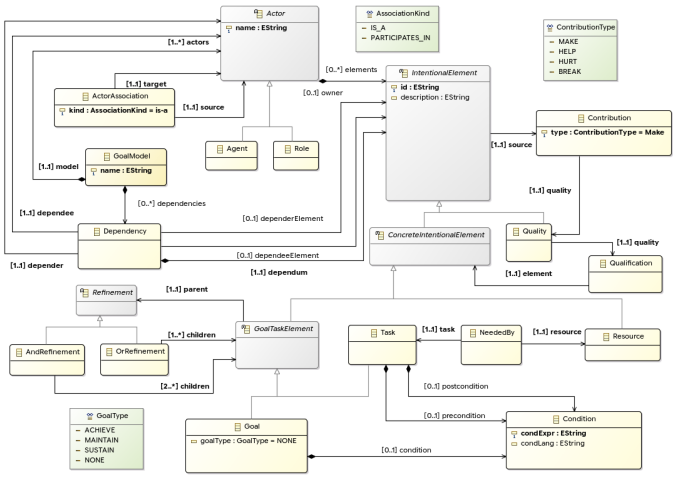


Fig. 4: iStar metamodel extended with conditions.

Definition 1 (Goal Model)**.** An extended iStar model is a tuple $\mathcal{G} = \langle A, N, R, \tau, \kappa_G \rangle$, where A is the set of Actors, N is the typed set of intentional elements, and $R \subseteq (A \cup N) \times N$ is the set of intentional relations. The partial function $\tau : N \rightharpoonup \mathcal{T}$, with $\mathcal{T} = \{Achieve, Maintain, Sustain, None\}$, assigns temporal semantics to goals, while κ_G provides state-dependent specifications for leaf elements.

Let $\Delta = \{\bot, ?, \top\}$ denote violated, undetermined, and satisfied, respectively. For an intentional element x, let $\hat{v}_i(x) \in \Delta$ denote the propagated value of x at step i of an execution, and let $v_i(x) \in \Delta$ denote its evaluated value. For a goal, $\top$ means that the goal is satisfied, whereas $\bot$ indicates a violation. For a task, $\top$ means that its pre- and postconditions hold, $\bot$ means that its precondition holds but its postcondition does not, and $?$ means that its precondition does not hold. The propagated value $\hat{v}_i(x)$ is determined according to Table I: for leaf elements, it is obtained directly from their associated conditions, while for non-leaf Goals it is derived from the evaluated values of their refinement or dependency elements. The goal is then evaluated from its propagated value, its

TABLE I: Instantaneous evaluation of iStar elements

Construct	$\hat{v}_i(x) = \top$	$\hat{v}_i(x) = \bot$
Leaf Goal g	$c_i(g)$	$\neg c_i(g)$
Leaf Task t	$pre_i(t) \wedge post_i(t)$	$pre_i(t) \wedge \neg post_i(t)$
AND refinement x	$\forall y \in Ch(x) : v_i(y) = \top$	$\exists y \in Ch(x) : v_i(y) = \bot$
OR refinement x	$\exists y \in Ch(x) : v_i(y) = \top$	$\forall y \in Ch(x) : v_i(y) = \bot$
Dependency x	$v_i(d(x)) = \top$	$v_i(d(x)) = \bot$

previous evaluated value, and its temporal type $\tau(g)$. A *None* goal follows the current propagated value; an *Achieve* goal retains satisfaction once achieved; a *Maintain* goal retains a violation once detected; and a *Sustain* goal records a violation if satisfaction is lost after being achieved. Tasks have no temporal interpretation, so $v_i(t) = \hat{v}_i(t)$. The state of the goal model at step i is the intentional marking $\mu_i : N \to \Delta$, where $\mu_i(x) = v_i(x)$.

B. Extension of the BPMN Model

To express constraints on state transitions for process executions, we extend the BPMN metamodel [10] with conditions attached to process elements, as shown in Fig. 5. Pre- and postconditions constrain the execution of *FlowElements*, while guards constrain the selection of *SequenceFlows*.

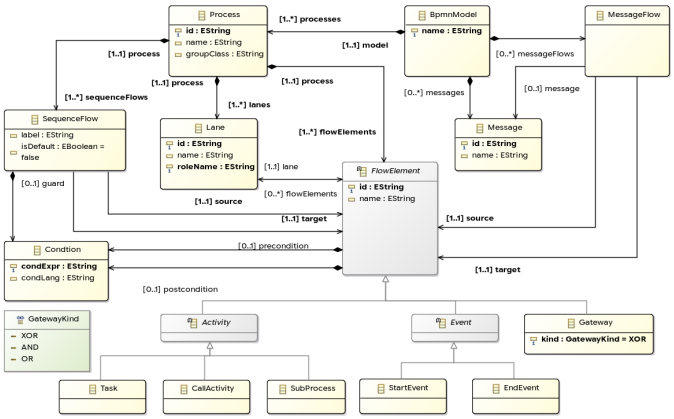


Fig. 5: BPMN metamodel extended with conditions.

Definition 2 (Process Model)**.** An extended BPMN model is a tuple $\mathcal{P} = \langle O, F, E, L, \kappa_P \rangle$, where O and F are the typed sets of organizational elements and flow elements. $E \subseteq F \times F$ contains the sequence flows, and $L \subseteq (F \cup O) \times O$ contains the organizational links. The component κ_P contains the state-dependent specifications associated with F and E.

A process state $q_i \subseteq F$ contains the flow elements that are enabled for execution at step i. A process transition is written $q_i \xrightarrow{a}_{\mathcal{P}} q_{i+1}$, where $a \in q_i$ is the executed flow element. The transition is valid when the precondition of a holds before execution, the guard of the selected outgoing sequence flow holds, and the postcondition of a holds after execution. The successor state q_{i+1} is obtained by removing a and adding the flow elements enabled through the selected sequence flow(s) in E.

C. Shared State and System Execution

The conformance between iStar and BPMN also requires the consistency between their organizational elements, such as the correspondence between iStar *Actors* and BPMN *Pools* or *Lanes* [6]. Therefore, a shared state model should capture the organizational structure together with the domain state. Our previous scenario-based approach used UML to represent system states [7], but UML does not explicitly capture the organizational semantics required here. We therefore introduce CSL, whose metamodel is shown in Fig. 6. In CSL, an *Entity* represents a passive domain concept, while an *OrgCtx* represents an organizational context that organizes *Entities*, *Roles*, and nested *OrgCtxs*. A *Role* represents an organizational position and follows the organizational semantics of MOISE+ [13]. At runtime, a *Role* instance is enacted by a

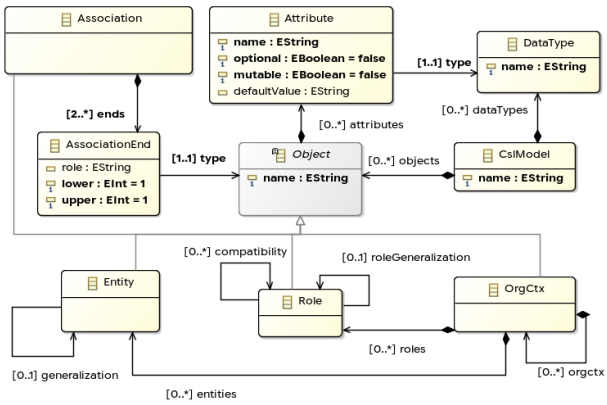


Fig. 6: CSL metamodel.

player, which may be either an atomic *Agent* or an *OrgCtx* instance, allowing an organizational context to act collectively in another context. The state associated with a *Role* belongs to the *Role* instance rather than to its player. *Role* generalization constrains specialized role enactment through the parent *Role*, whereas compatibility permits otherwise unrelated *Roles* to be enacted simultaneously.

Definition 3 (CSL Model)**.** Given an iStar model $\mathcal{G} = \langle A, N, R, \tau, \kappa_G \rangle$ and a BPMN model $\mathcal{P} = \langle O, F, E, L, \kappa_P \rangle$, a CSL model is $\mathcal{S} = \langle O_S, L_S, \Phi, H \rangle$, where O_S is the typed set of CSL objects, $L_S \subseteq O_S \times O_S$ contains their structural relations, Φ contains the OCL constraints, and $H \subseteq (A \times O_S) \cup (O \times O_S)$ contains the organizational correspondences with the iStar and BPMN models.

A CSL state σ records the current object instances, structural links, role enactments, and attribute values. A state is admissible when it satisfies Φ, written $\sigma \models \Phi$. $\Sigma_{\mathcal{S}}$ denotes the set of all admissible CSL states. Let $P_{\mathcal{P}}$ and $L_{\mathcal{P}}$ denote the subsets of O typed as Pool and Lane, and let $C_{\mathcal{S}}$ and $R_{\mathcal{S}}$ denote the subsets of O_S typed as OrgCtx and Role.

Definition 4 (Unified Model)**.** Let $\mathcal{G} = \langle A, N, R, \tau, \kappa_G \rangle$ be the iStar model, $\mathcal{P} = \langle O, F, E, L, \kappa_P \rangle$ the BPMN model, and $\mathcal{S} = \langle O_S, L_S, \Phi, H \rangle$ the CSL model. Their unified model $\mathcal{U} = \langle \mathcal{G}, \mathcal{P}, \mathcal{S} \rangle$ is well-formed iff

(i) $H \subseteq (P_{\mathcal{P}} \times C_{\mathcal{S}}) \cup ((A \cup L_{\mathcal{P}}) \times R_{\mathcal{S}})$;
(ii) $\forall p \in P_{\mathcal{P}},\ \exists! c \in C_{\mathcal{S}} \mid (p, c) \in H$;
(iii) $\forall x \in A \cup L_{\mathcal{P}},\ \exists! r \in R_{\mathcal{S}} \mid (x, r) \in H$.

For each $(p, o) \in H$ with $p \in P_{\mathcal{P}}$, let $K = Inst_{\sigma_0}(o)$ be the instances of o in the initial CSL state σ_0. Each $c \in K$ induces one runtime instance $\mathcal{P}[c]$ of the BPMN model and one runtime instance $\mathcal{G}[c]$ of the iStar model, both using c as their shared context. A start event determines the initial process state q_c of $\mathcal{P}[c]$.

Definition 5 (System State)**.** A system state is $s = \langle \sigma, \{\langle c, q_c, \mu_c \rangle \mid c \in K\} \rangle$, where $\sigma \in \Sigma_{\mathcal{S}}$ is the shared CSL state, $q_c \subseteq F$ is the process state of $\mathcal{P}[c]$, and $\mu_c : N \to \Delta$ is the intentional marking of $\mathcal{G}[c]$.

A system transition $s_i \xrightarrow{c:a} s_{i+1}$ executes a flow element $a \in q_c$ of $\mathcal{P}[c]$. Its precondition and the guard of the selected sequence flow are evaluated in σ_i with `self` $= c$, while its postcondition constrains the successor CSL state σ_{i+1}. The process state q_c advances according to the transition semantics of $\mathcal{P}$, while the process states of the other instances remain unchanged. Since σ is shared, the intentional marking of every $\mathcal{G}[c']$, $c' \in K$, is then updated from its previous marking using σ_{i+1} with `self` $= c'$.

Definition 6 (System Scenario)**.** A system scenario is a finite execution $\pi = \langle s_0, (c_1, a_1), s_1, \ldots, (c_n, a_n), s_n \rangle$ such that $s_{i-1} \xrightarrow{c_i:a_i} s_i$ for every $1 \leq i \leq n$. It is complete when every $\mathcal{P}[c]$, $c \in K$, has reached process completion in s_n.

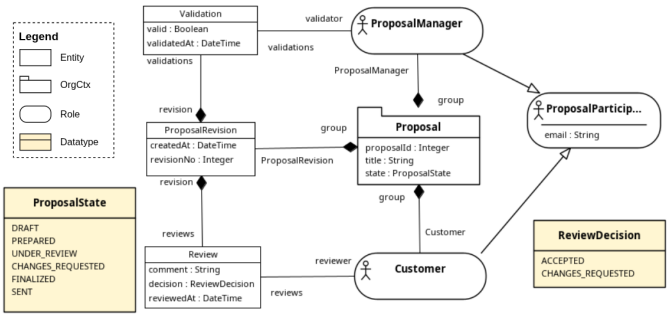


Fig. 7: The CSL model for *ProposalReview*.

Figure 7 presents the CSL model of *ProposalReview*. A `Proposal` captures the current state of the review process and reflects how the proposal evolves through successive revisions. The `ProposalManager` is responsible for producing new proposal revisions and validating them before they proceed further in the process. The `Customer` reviews the current revision and provides feedback, either accepting the proposal or requesting additional changes.

V. Checking the Conformance between iStar and BPMN Models

This section explains our method for checking conformance between iStar and BPMN models. The property *conformance* captures whether complete process executions preserve the intentions expressed in iStar, while *risk* captures whether any complete execution leads to a goal violation. *Strong conformance* holds when every complete execution preserves all root goals, *weak conformance* when this depends on the chosen execution path, and *non-conformance* when no complete execution satisfies all root goals. A model is risky whenever at least one complete execution leads to a goal violation; thus, a weakly conformant model may also be risky.

Definition 7 (Conformance and Risk)**.** Let $\mathcal{U} = \langle \mathcal{G}, \mathcal{P}, \mathcal{S} \rangle$ be a well-formed unified model, and let Π be the set of complete scenarios of $\mathcal{U}$. Let $N_R \subseteq N$ be the set of root goals. Let $\Pi_C = \{\pi \in \Pi \mid \forall c \in K, \forall g \in N_R, \mu_c^n(g) = \top\}$ be the set of conformant scenarios, and $\Pi_R = \{\pi \in \Pi \mid \exists c \in K, \exists g \in N_R, \mu_c^n(g) = \bot\}$ be the set of risky scenarios. For $\Pi \neq \emptyset$, $\mathcal{U}$ has

(i) *strong conformance* iff $\Pi_C = \Pi$;
(ii) *weak conformance* iff $\emptyset \subset \Pi_C \subset \Pi$;
(iii) *non-conformance* iff $\Pi_C = \emptyset$.

$\mathcal{U}$ is *risky* iff $\Pi_R \neq \emptyset$, and *non-risky* otherwise.

Algorithm 1: CHECKCONFORMANCE

Input : A unified model $\mathcal{U} = \langle \mathcal{G}, \mathcal{P}, \mathcal{S} \rangle$
Output: The conformance level and risk status of the unified model

$\Pi \leftarrow \emptyset$;
$Q \leftarrow$ INITIALSCENARIOS$(\mathcal{U})$;
while $Q \neq \emptyset$ **do**
  $\pi \leftarrow$ REMOVE(Q);
  $s \leftarrow$ LASTSTATE(π);
  if ISCOMPLETE(s) **then**
    $\Pi \leftarrow \Pi \cup \{\pi\}$;
  else
    foreach $(c, a) \in$ ENABLEDSTEPS(s) **do**
      foreach $s' \in$ EXECUTE(s, c, a) **do**
        add $\pi \cdot \langle (c, a), s' \rangle$ to Q;
return EVALUATECONFORMANCE(Π);

Algorithm 1 expresses conformance checking as scenario exploration over the unified model. It first obtains the initial scenarios from the admissible system states. Each scenario is then extended through its enabled BPMN execution steps until completion. The complete scenarios are collected in Π and evaluated according to Definition 7. The algorithm returns the conformance level together with the risk status.

VI. TOOL SUPPORT AND EXPERIMENTS

In this section, we present the implementation of a supporting tool and evaluate the proposed method through typical case studies. The evaluation illustrates the feasibility of the method and its ability to identify conformance and risk issues.

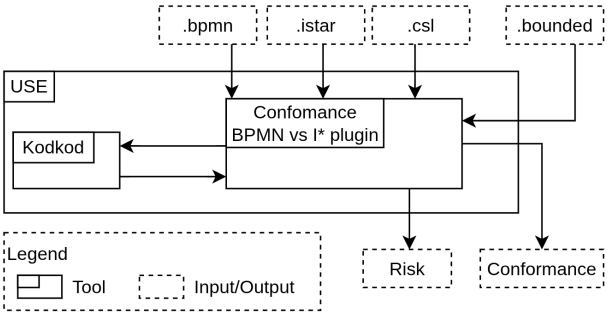


Fig. 8: Architecture of the analysis tool.

The method is implemented as a plugin for USE[1], reusing its UML/OCL modeling and evaluation facilities [14]. As shown in Fig. 8, the plugin takes iStar, BPMN, and CSL models together with a bounded configuration for finite-state exploration. Kodkod generates admissible CSL states satisfying the OCL constraints, while the plugin explores the executions of the unified model and evaluates the corresponding iStar states. From the resulting complete scenarios, the tool determines the conformance classification and risk status. For scenarios that fail to preserve the required intentions, it also provides the corresponding CSL-state sequence as a counterexample.

[1]https://github.com/vnu-sme/use4Goal

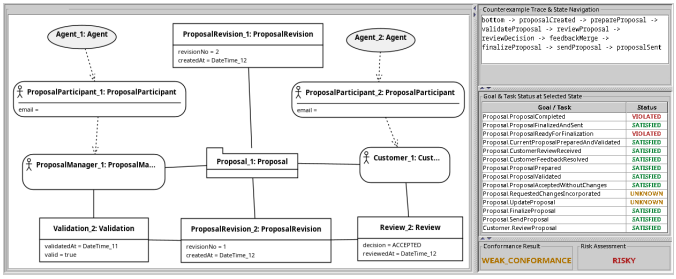


Fig. 9: The CSL model and analysis result for *ProposalReview*.

For *ProposalReview*, the tool explores the possible executions and records a counterexample trace when a complete execution violates a required intention. As shown in Fig. 9, the trace identifies the executed activities and allows navigation through the corresponding system states. For each selected state, the tool displays the CSL snapshot together with the current evaluation of iStar goals and tasks. The resulting conformance and risk verdicts are shown at the bottom of the interface.

TABLE II: Checking results across case studies

Case	Activities	Exec.	Time (s)	Result
ProposalReview	6	2	2.39	Weak / Risky
SalesForecast	28	258	2228.47	Weak / Non-risky
CustomerComplaint	13	120	475.24	Weak / Risky
StudentRegistration	54	126	1242.34	Weak / Non-risky
PurchaseOrder	52	30	203.31	Weak / Non-risky

The experiment evaluates five case studies adapted from Outmazgin et al. [8], with the results presented in Table II. The models span 6–54 BPMN activities and generate 2–258 complete executions. The results indicate that the checking cost is influenced by both the number of process activities and the number of execution paths generated by the process. For cases containing loops, we bound the number of loop traversals to one in order to limit the state space explored during verification.

DISCUSSION

The experiments show that the shared-state approach can reveal conformance violations caused by state changes during process execution. Because iStar and BPMN are both related to CSL, changes produced by the process can affect intention satisfaction without requiring direct goal–activity mappings. The *ProposalReview* case illustrates this situation, where the process can complete even though an earlier intention is no longer satisfied.

Since the unified model may generate infinitely many execution scenarios, the analysis relies on bounded verification over a finite number of snapshots. The specification of conditions and correspondences also depends on the modeler's domain knowledge and expertise. Moreover, the current evaluation is limited to a small set of case studies, so further experiments on larger and more complex systems are needed to assess both scalability and the representational capability of CSL. Future work will also investigate mechanisms for deriving mappings between iStar and BPMN elements through their common CSL references, enabling changes in one model to be traced to related elements in the other during model evolution.

VII. Related Work

A first research direction examines how goal and process models are connected through mappings, realization relations, and structural correspondences. GoBIS [2] integrates goal and process perspectives through metamodel integration and traceability, while Stra2Bis [15] and Guizzardi and Reis [16] relate business processes to strategic or organizational goals. Gröner et al. [4] and Eshuis and Ghose [17] use such relations to reason about structural consistency. In our approach, the iStar and BPMN models are independently mapped to CSL, allowing the two views to be developed separately.

A second research direction analyzes their consistency through execution behavior. Koliadis and colleagues [5], [6] relate goal satisfaction to the cumulative effects of activities along process trajectories. Their analysis is based on informal effect and fulfillment annotations, with formalization and automation identified as directions for further development. Our work follows the same execution-oriented perspective, but gives it formal shared-state semantics from which complete system scenarios can be generated and checked. Dang et al. [7] synchronize execution scenarios through a common conceptual model using TGG and OCL. Our approach instead uses CSL as the intermediate model to represent both organizational structure and domain state. Caballero-Villalobos et al. [3] represent process and iStar semantics as transition systems and analyze their combined behavior, characterizing compliance through reachability and stability of quality satisfaction. Our analysis instead evaluates state-dependent goals and tasks over complete system scenarios.

Our work is therefore positioned within execution-oriented conformance analysis between heterogeneous goal and process models. It combines independent mappings to a shared state model with systematic scenario exploration, while allowing multiple process instances to evolve the same system state during conformance analysis.

VIII. Conclusion

This paper has introduced a method for checking conformance between iStar goal models and BPMN process models. We proposed a unified model that integrates the two modeling languages and provides shared semantics, extending both iStar and BPMN with state-dependent specifications expressed in OCL. Based on this unified model, we defined CSL to represent the current state of an execution as a composition of the organizational and domain states. Execution scenarios are generated as sequences of state transitions through model finding and evaluated to determine whether the intentions expressed in the iStar model are satisfied by complete process executions. The analysis classifies the models as strong conformance, weak conformance, or non-conformance, together with the complementary notion of risk.

The method is supported by a plugin for USE and evaluated on *ProposalReview* and four additional process cases. In *ProposalReview*, the analysis identifies a risky execution in which a proposal is modified after validation without revalidation, resulting in weak conformance. The current evaluation is limited to a small set of cases, while the implementation relies on bounded state-space exploration and manual specification of conditions and correspondences. Further experiments on larger and more diverse models are needed to assess scalability and the representational capability of CSL. Future work will also investigate more efficient verification techniques and mechanisms for deriving mappings between iStar and BPMN to support change tracking during model evolution.

References

[1] A. Khalil and J. Dingel, "Optimizing the Symbolic Execution of Evolving Rhapsody Statecharts," in *Advances in Computers*. Elsevier, 2018, vol. 108, pp. 145–281.

[2] M. Ruiz, D. Costal, S. España, X. Franch, and Ó. Pastor, "GoBIS: An Integrated Framework to Analyse the Goal and Business Process Perspectives in Information Systems," *Information Systems*, vol. 53, pp. 330–345, 2015.

[3] J. Caballero-Villalobos, H. Baumeister, E. Paja, O. Kokoulina, and H. A. López, "Aligning Processes with High-Level Requirements: Goal-Model-Based Compliance Checking," *Information and Software Technology*, vol. 196, p. 108146, 2026.

[4] G. Gröner, M. Asadi, B. Mohabbati, D. Gašević, M. Bošković, and F. Silva Parreiras, "Validation of User Intentions in Process Orchestration and Choreography," *Information Systems*, vol. 43, pp. 83–99, 2014.

[5] G. Koliadis and A. K. Ghose, "Relating Business Process Models to Goal-Oriented Requirements Models in KAOS," in *Advances in Knowledge Acquisition and Management*, ser. Lecture Notes in Computer Science, vol. 4303. Berlin, Heidelberg: Springer, 2006, pp. 25–39.

[6] G. Koliadis, A. Vranesevic, M. Bhuiyan, A. Krishna, and A. K. Ghose, "Combining i* and BPMN for Business Process Model Lifecycle Management," in *Business Process Management Workshops*, ser. Lecture Notes in Computer Science, vol. 4103. Berlin, Heidelberg: Springer, 2006, pp. 416–427.

[7] D.-H. Dang, A.-H. Truong, and M. Gogolla, "Checking the Conformance between Models Based on Scenario Synchronization," *Journal of Universal Computer Science*, vol. 16, no. 17, pp. 2293–2312, 2010.

[8] N. Outmazgin, P. Soffer, and I. Hadar, "Leveraging Workarounds for a Problem-Focused Improvement of Business Processes," *Business & Information Systems Engineering*, vol. 68, no. 4, pp. 761–790, 2026.

[9] F. Dalpiaz, X. Franch, and J. Horkoff, "iStar 2.0 Language Guide," 2016.

[10] Object Management Group, "Business Process Model and Notation (BPMN), Version 2.0.2," Object Management Group, Tech. Rep. formal/2013-12-09, 2014, available: https://www.omg.org/spec/BPMN/2.0.2/.

[11] J. Warmer and A. Kleppe, *The Object Constraint Language: Getting Your Models Ready for MDA*, 2nd ed. Addison-Wesley Professional, 2003.

[12] H.-V. La Trinh and D.-H. Dang, "Transforming Operationalised iStar Models into Temporal Semantics and Event-B for Consistency Assurance," in *2025 17th International Conference on Knowledge and System Engineering (KSE)*. IEEE, 2025, pp. 250–255.

[13] J. F. Hübner, J. S. Sichman, and O. Boissier, "A Model for the Structural, Functional, and Deontic Specification of Organizations in Multiagent Systems," in *Advances in Artificial Intelligence (SBIA 2002)*, ser. Lecture Notes in Computer Science, vol. 2507. Springer, 2002, pp. 118–128.

[14] M. Gogolla, F. Hilken, and K.-H. Doan, "Achieving Model Quality through Model Validation, Verification and Exploration," *Computer Languages, Systems & Structures*, vol. 54, pp. 474–511, 2018.

[15] R. Noel, J. I. Panach, M. Ruiz, and O. Pastor, "Stra2Bis: A Model-Driven Method for Aligning Business Strategy and Business Processes," in *Conceptual Modeling*, ser. Lecture Notes in Computer Science, vol. 13607. Cham: Springer International Publishing, 2022, pp. 255–270.

[16] R. S. S. Guizzardi and A. N. Reis, "A Method to Align Goals and Business Processes," in *Conceptual Modeling*, ser. Lecture Notes in Computer Science, vol. 9381. Cham: Springer International Publishing, 2015, pp. 79–93.

[17] R. Eshuis and A. Ghose, "Consistency Checking of Goal Models and Case Management Schemas," in *Business Process Management Forum, BPM 2021, Proceedings*, ser. Lecture Notes in Business Information Processing, vol. 427. Cham: Springer International Publishing, 2021, pp. 54–70.